

AgentHound: graph-based attack-path analysis for multi-agent LLM architectures

Pablo González

CDO, Telefónica
Madrid, Spain

Fran Ramírez

pablo.gonzalezperez@telefonica.com
franciscojose.ramirezvicente@telefonica.com

Abstract- Multi-agent systems built on large language models compose agents, tools, external data, shared memory and outputs into pipelines whose risk does not live in any single component. A tool that is safe in isolation becomes dangerous once an untrusted input can reach it and its result can leave the trust boundary. AgentHound models such an architecture as a typed directed graph, following the BloodHound principle that capabilities are edges and risk-relevant properties live on nodes. It accepts a YAML description of the architecture, detects the source framework, normalizes it into an intermediate representation, builds a NetworkX MultiDiGraph, and runs a deterministic rule engine that emits prioritized findings over nodes, edges and paths. The analysis uses no language model: the same input always yields the same findings. Each finding carries a rule identifier, the offending path, an additive risk score, evidence, recommended controls, and mappings to OWASP and MITRE ATLAS. A control simulator then clones the graph, applies chosen controls, cuts or weakens the affected edges, recomputes the findings, and reports the reduction in risk. This paper describes the model, the detection methodology, the scoring function, the control simulation and the implementation, and it walks through a recruiting-pipeline case study run through the tool.

Keywords- LLM agents; attack paths; threat modeling; graph analysis; prompt injection; security controls.

I. INTRODUCTION

A single agent that can read a customer record is not, by itself, a breach. The same agent becomes a data-exfiltration path once an attacker-controlled email can steer it and the agent can also send mail to an outside address. Risk in agentic systems is a property of composition: the untrusted source, the sensitive read, the missing approval gate and the external sink only add up to an incident when they line up on one path. Reviewing each tool on its own misses this, because every tool passes its local check while the dangerous combination goes unexamined.

AgentHound targets that gap. It reads an architecture description, turns it into a graph, and searches for the paths where an untrusted input can drive a sensitive action across a trust boundary with no control in the way. The design borrows from BloodHound^[1], the tool that reframed Active Directory auditing around attack paths rather than isolated misconfigurations. In AgentHound the nodes are the parts of an agentic system and the edges are the capabilities that connect them; the sensitivity, externality and approval state that decide whether a path is dangerous are read from node properties, not encoded in the edge names. The engine is deterministic and calls no language model, so its findings are reproducible and auditable.

This is threat modeling, not vulnerability scanning, and the distinction matters. A scanner inspects components for known weaknesses, an outdated dependency, an injectable parameter, a permission left too wide, and reports each one as a defect to patch. AgentHound assumes every component is individually correct and asks a harder question: given how the pieces are wired, where can an untrusted input drive a sensitive action across a trust boundary? A finding here is not a bug in a library. It is a path the architecture permits by design. The fix is rarely a patch; it is a control placed on the right edge, which is why the tool pairs every finding with the mitigation that breaks it.

Contributions.

This work makes four practical contributions. It defines a framework-neutral intermediate representation for multi-agent architectures with seven node types and a fixed vocabulary of neutral capability verbs. It specifies a deterministic rule engine over nodes, edges and paths, with an additive, bounded risk score and explicit severity bands. It provides a control simulator that measures the before-and-after effect of concrete mitigations by cutting or weakening edges and recomputing findings. It ships a working backend with adapters for CrewAI, Dify, LangGraph and a generic format, plus catalogs that keep the capability set, node defaults, controls and rules declarative and auditable.

II. BACKGROUND AND RELATED WORK

BloodHound^[1] established graph attack-path analysis as a way to reason about privilege in complex systems: model the environment as a directed graph, then look for reachable paths from a low-privilege foothold to a high-value target. AgentHound applies the same idea to LLM agents. The difference is what the nodes and edges mean. Here the nodes are agents, inputs, tools, data assets, outputs, memory stores and controls, and the edges are capabilities such as read, fetchWeb, send and delegateTo.

The risk taxonomy comes from two public sources. The OWASP work on LLM and agentic-application risks^{[2], [3]} supplies the categories AgentHound reports against, including prompt injection (LLM01), sensitive information disclosure (LLM02) and excessive agency (LLM06), together with agentic threats such as tool misuse and identity abuse. MITRE ATLAS^[4] supplies adversary technique identifiers, for example AML.T0051 for prompt injection and AML.T0086 for exfiltration through an agent's own tools. AgentHound

does not restate these frameworks; it maps each rule onto them so a finding can be traced to a recognized technique. What is new here is the combination: a framework-neutral graph model of agentic systems, a path-level rule engine, and a control simulator that quantifies how much risk a given mitigation removes.

Recent tooling for agentic systems leans toward static scanning: parse the code or the framework configuration, enumerate the components and their known weak points, and return a list of issues to fix. That work is useful, but it answers a different question. It tells you whether a component is weak; it does not tell you whether the composition is exploitable. Two systems can pass the same component checks and still differ entirely in whether an untrusted email can reach an external send. AgentHound operates on the composition rather than the component, on the graph rather than the file, because that is the level at which agentic incidents actually happen. Some rules do read configuration a scanner would recognize, such as a disabled sandbox or an unscoped delegation. The difference is that a weak property counts only when it lies on a reachable path from an untrusted source to a sensitive or external target. A scanner reports the disabled sandbox on its own; AgentHound raises it only when the composition makes it dangerous. Property plus reachability, not property alone.

III. THREAT MODEL

AgentHound reasons over three trust regions: what lies outside the system and is untrusted, the internal agents and tools, and the data, memory and outputs they touch. Inputs default to untrusted and outputs default to external, because under-reporting on the exposure boundary is the dangerous direction. The adversary is assumed to control the content of an untrusted input, for example an incoming email or a fetched web page, and to have no direct access to the internal components. The adversary's goal is to make an agent take an action it should not: read sensitive data and send it out, act on malicious instructions hidden in fetched content, or poison shared state that another agent later trusts.

Three path shapes capture the bulk of what the tool looks for. The first is a full exfiltration path: an untrusted input reaches an agent that reads a high-sensitivity asset and ends at an external output with no approval gate. The second is indirect injection through fetched web content: an agent retrieves external content, the hostile text enters the model context, propagates to a second agent, and drives an external action. The third is memory poisoning: an agent writes attacker-influenced data into a shared memory store that other agents read without validation. Figure 1 shows the node types, capability verbs and scoring inputs that these shapes are built from.

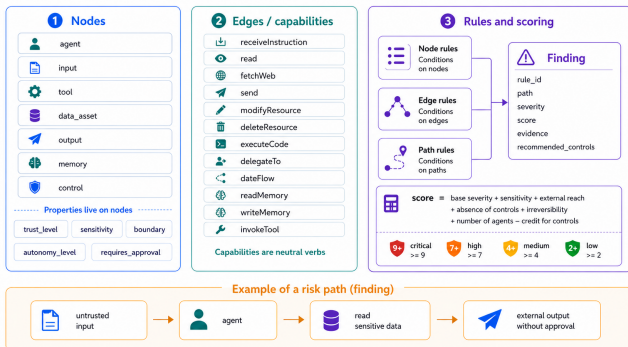


Figure 1. Analysis and risk-detection model: node types, neutral capability edges, and the node, edge and path rules that feed a finding.

IV. SYSTEM DESIGN AND ARCHITECTURE

The backend is layered. A REST API exposes the analysis and simulation endpoints. A parser-and-normalization layer loads the YAML, detects the framework, runs the matching adapter, and produces the intermediate representation. An analysis core builds the graph, traverses it, runs the rules and computes scores. A controls layer recommends and simulates mitigations. A persistence layer keeps results in memory and on disk as JSON. Four YAML catalogs sit beside these layers and hold the capability vocabulary, the node property defaults, the control definitions and the rule metadata. Figure 2 shows this internal structure.

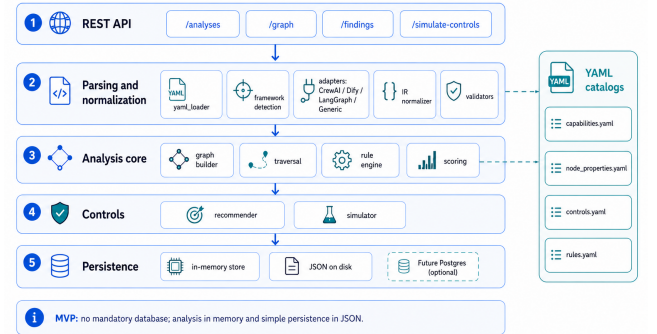


Figure 2. Internal backend architecture: REST API, parser and normalization, analysis core, controls, persistence, and the four YAML catalogs.

End to end the pipeline runs in nine stages: a YAML architecture is ingested; the framework is detected; the input is parsed and adapted; a normalized intermediate representation is produced; the graph is built as a NetworkX MultiDiGraph; the rule engine evaluates node, edge and path rules; findings are prioritized by score, severity and evidence; controls are recommended; and a simulation returns a mitigated graph with the broken paths marked. The MVP keeps analyses in memory and persists them as JSON; a database is optional and reserved for multi-user use. Figure 3 shows the flow and the outputs handed to the frontend.

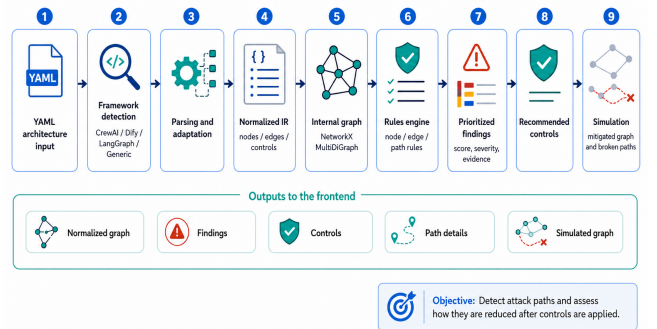


Figure 3. General backend flow, from the YAML architecture through framework detection, normalization, graph construction, rules, findings and simulation, to the outputs consumed by the frontend.

V. DATA MODEL AND INTERMEDIATE REPRESENTATION

The intermediate representation is one flat list of nodes and one flat list of edges, plus a list of declared controls and light

metadata. Each node has an id, a type, a human-readable name and a free-form properties map. Each edge has an id, a source, a target, a neutral capability verb and a properties map. The seven node types are agent, input, tool, data asset, output, memory and control. The capability verbs are a closed set: receiveInstruction, read, fetchWeb, send, modifyResource, deleteResource, executeCode, delegateTo, dataFlow, readMemory, writeMemory and invokeTool, with search, extractInformation and createResource added as refinements in the catalog. The rule against encoding risk in an edge name is strict. There is no readSensitive or sendExternal verb; sensitivity and externality are read from the target node instead.

Risk-relevant properties live on the nodes. The properties the rules and scoring read include `trust_level`, `sensitivity`, `boundary`, `side_effect`, `requires_approval`, `autonomy_level` and `approval_gate_present`. When a property is missing, the normalizer applies a cautious default so the analysis does not under-report on the trust or exposure boundary. Table 1 lists the defaults that matter most.

Table 1. Safe defaults applied by the normalizer when a property is missing.

Node type	Property	Safe default
input	<code>trust_level</code>	untrusted
output	<code>boundary</code>	external
output	<code>approval_gate_present</code>	false
agent	<code>autonomy_level</code>	full_auto
agent	<code>guardrails</code>	[] (none)
data_asset	<code>sensitivity</code>	medium
tool	<code>requires_approval</code>	false
tool	<code>side_effect</code>	false

The scoring enumerations are ordered so rules can compare levels: `trust_level` runs `untrusted`, `semi_trusted`, `internal_logic`, `trusted_core`; `sensitivity` runs `public`, `low`, `medium`, `high`, `critical`; `boundary` runs `internal`, `perimeter`, `external`; and `autonomy_level` runs `supervised`, `semi_auto`, `full_auto`. A data asset counts as sensitive for path detection when its sensitivity is high or critical.

VI. DETECTION METHODOLOGY

The rule engine evaluates three kinds of rule. Node rules test a condition on a single node and its immediate neighbourhood, for example whether an untrusted input can reach a read over a high-sensitivity asset. Edge rules test a condition on a capability edge in the context of its endpoints. Path rules walk the graph and test a condition on a whole path, for example an untrusted input that reaches an external output through a sensitive read. Path traversal is bounded by a maximum depth of eight hops to keep the search tractable on large graphs.

Every match produces a finding. A finding records the `rule_id`, a title, the severity, the numeric score, a category, the ordered node path and edge ids, a derivation string, an evidence map, the recommended controls, and the OWASP and MITRE ATLAS mappings. The analyzed code snapshot registers seven rules. Table 2 lists them with their base severity and standards mappings.

Table 2. Detection rules in the analyzed code snapshot, with base severity and current-standard OWASP mappings.

Rule	Base	OWASP LLM 2025	OWASP Agentic 2026
FH-001	critical	LLM02, LLM06	ASI02
FH-002	high	LLM02	ASI01
FH-003	high	LLM06	ASI02

FH-006	high	LLM01	ASI01
FH-010	high	LLM06	ASI05
FH-011	high	LLM06	ASI02
FH-014	high	LLM06	ASI03

The deployed build adds further path rules beyond this snapshot, among them FH-007 (a multi-agent chain reaching an external output with no intermediate control) and FH-023 (unscoped delegation between agents with disjoint capabilities). FH-007 maps to insecure inter-agent communication (ASI07) and FH-023 to identity and privilege abuse (ASI03). Both appear in the case study of Section 9.

Mappings follow specific standard versions. The OWASP Top 10 for LLM Applications 2025 (LLM01:2025 to LLM10:2025, published on 18 November 2024) supplies the LLM categories, of which three recur here: LLM01 Prompt Injection, LLM02 Sensitive Information Disclosure and LLM06 Excessive Agency. The OWASP Top 10 for Agentic Applications 2026 (ASI01:2026 to ASI10:2026, published in December 2025) supplies the agentic categories: ASI01 Agent Goal Hijack, ASI02 Tool Misuse and Exploitation, ASI03 Identity and Privilege Abuse and ASI05 Unexpected Code Execution. Where a MITRE ATLAS technique is precise it is named in full: FH-006 is Indirect Prompt Injection via Retrieved Content (AML.T0051.001), FH-002 is LLM Data Leakage (AML.T0057), and the exfiltration in FH-001 and FH-003 is Exfiltration via AI Agent Tool Invocation (AML.T0086) or External Harms (AML.T0048). A rule maps to a category when the path it detects is a concrete instance of it: FH-003, for example, reaches an external actioning capability from untrusted input, which is excessive agency (LLM06) realized through tool misuse (ASI02).

Scoring.

The score is additive and then clamped to the range zero to ten. It sums a base value for the rule severity, a weight for the maximum data sensitivity on the path, a weight for external reach, a weight for each absent control class, a penalty when the action is irreversible, and a weight for the number of agents on the path; it then subtracts a credit for controls already present. Table 3 gives the component weights.

Table 3. Risk-score components and their weights.

Component	Values
Base severity	critical 8.0, high 6.0, medium 4.0, low 2.0
Data sensitivity	public 0.0, low 0.3, medium 0.8, high 1.8, critical 2.5
External reach	internal 0.0, perimeter 1.0, external 2.0
Control absence	approval 1.0, validation 0.8, allowlist 0.7
Irreversibility	reversible 0.0, irreversible 1.0
Agent count	one 0.0, two 0.3, three or more 0.8
Mitigation credit	1.5 per mitigated hop (subtracted)

The resulting score maps to a severity band: critical at nine or above, high at seven or above, medium at four or above, low at two or above, and informational below two. Consider FH-003 in the recruiting pipeline of Section 9, where an untrusted web form reaches an external email send with no approval gate across three agents. The base for a high-severity rule is 6.0, external reach adds 2.0, the missing approval adds 1.0, and three agents on the path add 0.8; sensitivity does not enter, because the rule concerns an external action rather than a sensitive read. The sum is 9.8, which lands in the critical band even though the base severity is high. That is the exact value the tool reports for the path (Figure 7). The clamp does bind on the most severe findings: when a critical base, a high-

sensitivity read, an external sink and several agents stack on one path, the raw sum exceeds ten and is reported as 10.0.

VII. CONTROL SIMULATION

A control is a first-class mitigation with a declared effect. Each control names the capability kinds it acts on and whether its effect is total or partial. A total-effect control marks every affected edge as blocked and cuts it from the recomputed graph, so any path through that edge disappears. A partial-effect control marks the affected edges as mitigated: the hop is kept, but every surviving finding whose path crosses a mitigated edge loses a fixed credit of 1.5 per crossing from its score. Only blocked edges are removed from traversal.

The simulator clones the intermediate representation so the stored analysis is untouched, adds the applied controls as control nodes, cuts the blocked edges, and re-runs the same rule engine. It then returns a before summary and an after summary, each with a finding count, a critical count and a maximum score; a risk reduction expressed in absolute and percentage terms; the list of broken paths with the control that broke each one; and the simulated graph with blocked and mitigated edges marked. The current catalog holds controls such as humanInTheLoop on actioning capabilities like send and executeCode, contentValidation on fetchWeb, delegationScoping on delegateTo, sandboxIsolation on executeCode, and partial controls such as leastPrivilege, inputSanitization, outputFiltering and piiRedaction. Figure 4 shows the analysis and simulation endpoints and an illustrative before-and-after comparison.

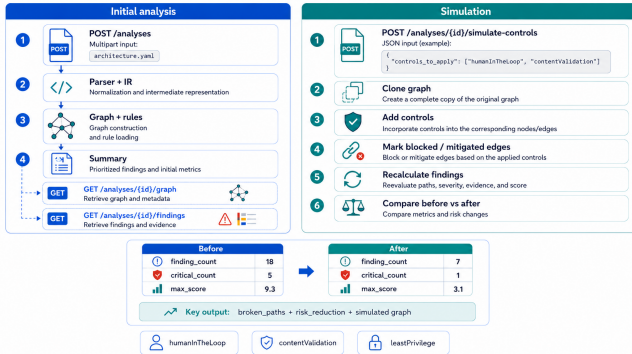


Figure 4. Analysis and control simulation. The before-and-after counts shown here are illustrative values from the design material.

VIII. IMPLEMENTATION

The backend is written in Python 3.11 with FastAPI for the API, Pydantic v2 for the models, NetworkX for the graph, and PyYAML for loading. Framework detection is a small heuristic applied when the caller does not name a framework: a document with both agents and tasks keys is treated as CrewAI; a document whose workflow object contains a graph is treated as Dify; a document with a state_graph or a top-level graph key is treated as LangGraph; anything else is handled by the generic adapter. Each adapter extracts agents, tools, inputs, outputs, data assets and any memory, creates the edges, applies the safe defaults and validates the intermediate representation.

Four YAML catalogs keep the analysis declarative. The capabilities catalog documents each verb and the node kinds it usually connects. The node-properties catalog lists the risk-relevant properties per type and their defaults. The controls catalog defines each control, the capability kinds it breaks or

mitigates, its effect, and presentation hints used by the recommender. The rules catalog carries the presentation metadata, base severity, recommended controls and standards mappings for each rule, while the detection logic lives in the node and path rule modules and is keyed by rule id. Results are held in an in-memory repository and can be persisted as JSON, with no database required for the MVP. The backend source is public [5].

Deterministic analysis, no model in the loop.

The analysis uses no language model. Framework detection is a set of key checks over the YAML; the adapters copy fields into the intermediate representation; the capability catalog assigns the neutral verb for each connection; and traversal, the rule predicates and the additive score are ordinary code. The same input produces the same findings every time, which is what a security review needs and what a model in the loop would not give. The systems under analysis run on LLMs, but AgentHound reasons about their structure without calling one. Where the extraction from a terse YAML is ambiguous, a person corrects it directly: the agent editor exposes the same neutral-verb catalog the analysis uses, so a reviewer can add or remove a capability and rerun. Figure 5 shows that editor.

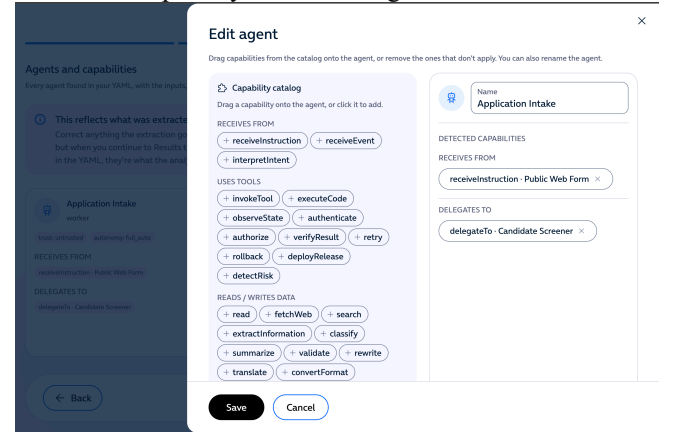


Figure 5. Agent editor. Capabilities are assigned from a fixed neutral-verb catalog and shown as detected from the YAML; a reviewer edits them by hand and reruns. The extraction and analysis are deterministic, with no model involved.

IX. CASE STUDY: THE HIRESMART RECRUITING PIPELINE

The running example is HireSmart, a recruiting pipeline. A public web form feeds an intake agent, which delegates to a screener that reads a candidate ATS database and CV attachments, both high sensitivity, and to a communicator that emails candidates. Every input is untrusted, and the candidate email is external with no approval gate.

9.1 Findings through the CrewAI adapter.

Analyzed through the CrewAI adapter, the pipeline resolves to three agents and the tool reports seventeen findings, one of them critical, with a maximum score of 9.8 (Figure 7). The ranked attack paths are led by FH-003 at 9.8, an actionable external output reachable from untrusted input, followed by FH-002 at 8.8, a sensitive read reachable from untrusted input; FH-007 at 8.8, a multi-agent chain with no intermediate control; FH-014 at 7.1, privilege escalation through unscoped delegation; and FH-023 at 5.1, unscoped delegation between agents with disjoint capabilities. FH-007 and FH-023 come from the additional path rules noted in Section 6. The recommended choke point is context isolation on the Candidate Screener, which the tool estimates removes 53.5

points, about 51 percent of the accumulated risk. Figure 6 shows the FH-003 route drawn on the graph, from the public web form through the two agents to the external send.

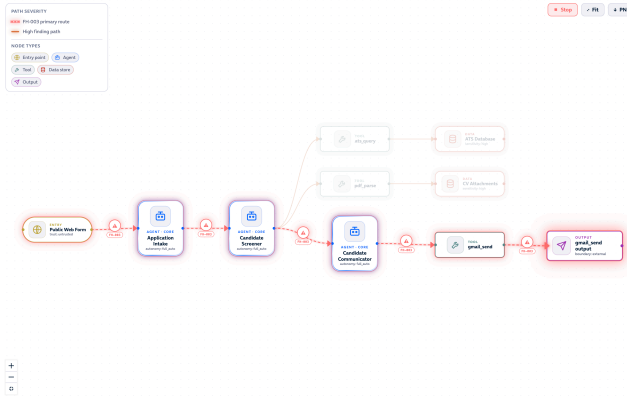


Figure 6. HireSmart through the CrewAI adapter. The FH-003 route runs from the untrusted web form through the intake and screener agents to the external gmail_send output.



Figure 7. Risk metrics for the CrewAI run: one critical finding, a maximum score of 9.8, three agents, and the recommended choke point on the Candidate Screener.

9.2 Simulating the controls.

Simulating the recommended controls on the CrewAI run moves the metrics from seventeen findings, one critical and a maximum score of 9.8, to ten findings, none critical and a maximum of 7.6, a 52.9 percent reduction of about 55 points (Figure 8). The tool attributes the largest single drop to context isolation on the Candidate Screener at 53.5 points, then delegation scoping on the Candidate Communicator at 23.7 points, and human approval on the gmail_send action at 18.6 points. Human approval on the send is the cheap, total control: it cuts the send edge and removes FH-003 and FH-007 at once. This is the choke-point pattern the whole model is built to surface, a single control on the right edge that breaks several paths. These per-control figures overlap and are not additive. Each is computed as if that control were applied alone, so two controls that both break FH-003 both claim its points: the estimate for context isolation on the Candidate Screener and the one for delegation scoping on the same agent cover the same set of findings, not two separate sets, and adding them would double count. The before-and-after summary is the de-duplicated result of applying the chosen set together, a single reduction of about 55 points or 52.9 percent, which is why it is smaller than the sum of the individual estimates, and why the choke-point figure quoted as 51 percent for one control in Figure 7 differs from the 52.9 percent of the applied package in Figure 8.

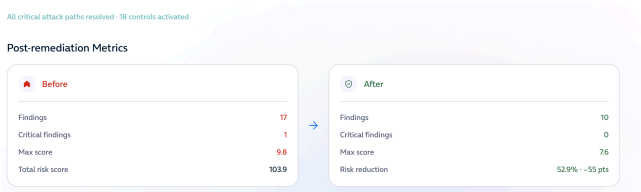


Figure 8. Before-and-after metrics for the CrewAI run after simulating the recommended controls: findings drop from seventeen to ten, critical findings from one to zero, and risk by 52.9 percent.

X. LIMITATIONS

AgentHound performs static analysis of a declared architecture. It reasons about what a system can do, not what it does at run time, so it cannot confirm that a path is exploitable in practice or observe a live prompt injection. Its accuracy depends on the fidelity of the YAML: an architecture that omits a tool or misstates a boundary yields an incomplete graph. The shape of the input matters too: the way an adapter lays out nodes and edges determines which paths are expressible, so the same pipeline can yield different findings depending on how it is described. Adapter coverage is limited to CrewAI, Dify, LangGraph and the generic format, and a framework outside that set falls back to the generic adapter with reduced structure. The rules and the scoring weights are heuristics chosen for a demonstrable, conservative result; the additive score with a hard ceiling is easy to read but compresses distinctions among the most severe findings. There is no runtime evidence linking a finding to an observed event.

One consequence follows from the category. AgentHound does not enumerate vulnerabilities or assign CVEs, so a clean report is not a claim that the components are free of defects. It states only that the modelled attack paths are broken or absent. Component-level scanning and threat modeling answer different questions, and a serious review wants both.

XI. FUTURE WORK

Several extensions follow from the current design. Broader framework support would add adapters and improve detection beyond the heuristic sketch. Runtime correlation would connect a static path to telemetry from a running system, turning a possible path into an observed one. A move from in-memory and JSON persistence to a database would support multi-user and historical analysis. A richer control catalog and a recommender that ranks controls by measured risk reduction would tighten the link between a finding and the cheapest mitigation that removes it. The rule set is already growing past the snapshot documented here, and completing a memory-poisoning family would close the gap between the documented path shapes and the rules the engine ships.

XII. CONCLUSION

Risk in multi-agent LLM systems is compositional, and a per-tool review does not see it. AgentHound reframes the problem as attack-path analysis over a typed graph, in the spirit of BloodHound: capabilities are edges, risk properties live on nodes, and a deterministic rule engine, with no model in the loop, finds the paths where an untrusted input can drive a sensitive action across a trust boundary without a control in the way. Findings carry a bounded score, evidence and standards mappings, and a control simulator measures how much risk a given mitigation removes by cutting or weakening edges and recomputing the findings. The recruiting-pipeline case study shows the whole loop: the tool ranks the paths, names a choke point, and quantifies the risk removed once the control is applied.

REFERENCES

- [1] BloodHound, SpecterOps: graph-based attack-path analysis for Active Directory and Entra ID. <https://github.com/SpecterOps/BloodHound>
- [2] OWASP Top 10 for Large Language Model Applications 2025 (LLM01:2025 to LLM10:2025), OWASP GenAI Security Project, published 18 November 2024. <https://genai.owasp.org/llm-top-10/>
- [3] OWASP Top 10 for Agentic Applications 2026 (ASI01:2026 to ASI10:2026), OWASP GenAI Security Project, published December 2025. <https://genai.owasp.org/initiatives/agentic-security-initiative/>
- [4] MITRE ATLAS, Adversarial Threat Landscape for Artificial-Intelligence Systems, matrix v5.x, 2025. <https://atlas.mitre.org/>
- [5] AgentHound, Telefónica: backend source repository. <https://github.com/Telefonica/AgentHound>